



- ▶ Uso básico de git
- ▶ GitHub enlazado a VS Code

Bibliografía

- Git Software. *Git Documentation*: <https://git-scm.com/doc>
- *Introducción a la documentación de GitHub*: <https://docs.github.com/es/get-started>
- *Introduction to Git and GitHub for Python Developers*:
<https://realpython.com/python-git-github-intro/>

Git

Git es un sistema de control de versiones que permite a los desarrolladores rastrear cambios en el código fuente durante el desarrollo de algún código. Fue creado por Linus Torvalds en 2005 para ayudar en el desarrollo del kernel de Linux. Git permite, a través de GitHub, sincronizar repositorios entre múltiples desarrolladores trabajar en un proyecto simultáneamente, manteniendo un historial completo de cambios y facilitando la colaboración. Sin embargo, hay que enfatizar que no es necesario usar GitHub para usar Git. El flujo típico de trabajo con Git es el siguiente:

1. Iniciar un repositorio local en su máquina

```
1 git init
```

2. Añadir los textos y archivos de código al repositorio local

```
1 git add formas_diferenciales.tex
```

En este momento, te debió aparecer un error. Esto se debe a que antes de agregar los archivos, hay que configurar Git. Para ello, ejecutas

```
1 git config --global user.name "usuario"  
2 git config --global user.email "correo@itm.edu.co"
```

Como recomendación, es ideal que el correo sea el mismo con el que se registrarán en GitHub para que la plataforma asocie correctamente los commits a su perfil.

3. Hacer cambios en los archivos que quieras. Luego para ver qué has modificado, ejecutas

```
1 git status
```

En este caso, git te mostrará algo como modified: formas_diferenciales.tex

4. Añades los cambios (puedes usar `git add .` para añadir todos los cambios que hiciste)

```
1 git add formas_diferenciales.tex
```

5. Comentas los cambios para tener control de las versiones

```
1 git commit -m "Se agregan ejemplos de formas diferenciales exactas al  
2 archivo formas_diferenciales.tex"
```

Con esto ya puedes modificar los archivos y saber que has cambiado o que versiones tienes y volver a las versiones anteriores. Comandos como `git diff`, `git log` sirven para ello.

En los proyectos, usualmente tenemos muchos archivos basura que no deseamos seguir. Para ignorarlos, existe un archivo que podemos crear antes del primer `git add`. Abrimos un archivo `emacs .gitignore` y agregamos las cosas que no queramos seguir, por ejemplo

```
1 # Entornos virtuales  
2 __pycache__/  
3 *.pyc  
4 .ipynb_checkpoints/  
5 *.venv/  
6  
7 # Archivos privados  
8 2026-2/  
9  
10 # vs code  
11 .vscode/  
12  
13 # Archivos temporales de sistema y editores  
14 .DS_Store  
15 *.swp  
16 *~  
17 *.aux  
18 *.log  
19 *.fls  
20 *.fdb_latexmk  
21 *.synctex.gz  
22 *.out  
23 *.toc  
24 *.nav  
25 *.snm  
26 *.bbl  
27 *.blg
```

Git permite el uso de ramas para experimentar con nuestros proyectos y si todo sale bien, juntarlo con la rama principal.

```
1 # Crear y cambiar a una nueva rama de experimento  
2 git checkout -b hipotesis-Rham  
3 # Listar ramas existentes y verificar la posicion actual  
4 git branch
```

Una vez que el experimento ha sido validado, los cambios se integran a la rama principal. Si el experimento falla, la rama puede ser eliminada sin dejar residuos en el código base.

```
1 # Regresar a la rama principal e integrar cambios
2 git checkout main
3 git merge hipotesis-Rham
4 # Eliminar la rama experimental tras la fusion
5 git branch -d hipotesis-Rham
```

Por otro lado, en ocasiones es importante trabajar con varias personas al tiempo y que cada una haga modificaciones a los archivos de tal forma que las otras personas vinculadas al proyecto los puedan tener. Para esto, se suele usar la plataforma GitHub y en este caso hay un par de comandos adicionales a tener en cuenta, `git pull` y `git push`. En nuestro curso no entraremos en los detalles del uso de `git`, por lo que queda de interés a la persona lectora, la profundización de lo que aquí se ha expuesto. Sin embargo, las tareas del curso serán entregadas por GitHub, entonces explicaremos como crear una cuenta, clonar el proyecto del curso y entregar las tareas.

Creación de cuenta en GitHub

Para comenzar, ingrese a la página de GitHub <https://github.com> y haga clic en **Sign up**. Complete el formulario de registro usando preferiblemente su correo institucional. Siga las instrucciones en pantalla para verificar su cuenta desde el enlace enviado a su correo electrónico.

Vincular GitHub con Visual Studio Code

Para trabajar directamente desde el editor sin necesidad de ingresar comandos por terminal en cada acción:

1. Abra **Visual Studio Code**.
2. En la barra lateral izquierda, haga clic en el icono de **Cuentas** (esquina inferior izquierda) o diríjase a la pestaña de **Control de código fuente**. El atajo de teclado (`Ctrl + Shift + G`).
3. Seleccione la opción **Sign in with GitHub**.
4. Se abrirá una ventana en su navegador web solicitando autorización para conectar VS Code con su cuenta de GitHub. Confirme el acceso.

Guía para la entregas de Tareas vía VS Code y GitHub

En nuestro curso, las tareas se entregarán a través de GitHub utilizando una combinación entre la interfaz web y Visual Studio Code. A continuación se detallan los pasos para configurar su entorno y realizar las entregas.

1. Crear un Fork (Copia personal del repositorio del curso)

Un **Fork** es una fotocopia exacta del repositorio del curso en su cuenta personal. Esto garantiza que usted pueda trabajar libremente sin modificar el archivo original.

- 1) Ingrese al enlace del repositorio del curso [dando clic aquí](#).
- 2) En la esquina superior derecha de la página de GitHub, haga clic en el botón **Fork** y luego en **Create fork**.
- 3) Verifique que la página web ahora muestre la URL con su nombre de usuario (ejemplo: `github.com/su_usuario/Fisica_Computacional_1`).

2. Clonar SU Fork en VS Code

- 1) En la página web de **su Fork**, haga clic en el botón verde **Code** y copie la URL que aparece bajo la pestaña **HTTPS**.
- 2) En VS Code, presione `Ctrl + Shift + P` para abrir la paleta de comandos.
- 3) Escriba `Git: Clone` y presione **Enter**.
- 4) Pegue la URL de su Fork y presione **Enter**.
- 5) Seleccione una carpeta en su equipo para guardar el proyecto y confirme haciendo clic en **Abrir**.

3. Resolver las Tareas y Sincronizar Cambios

- 1) Resuelva los ejercicios editando o creando los archivos necesarios en VS Code (scripts `.py`, documentos `.tex`, etc.). Guarde sus avances.
- 2) Diríjase al panel de **Control de código fuente**. El atajo en teclado es (`Ctrl + Shift + G`).
- 3) Pase el cursor sobre la sección *Changes* y haga clic en el icono **+** para preparar todos los archivos modificados.
- 4) Escriba un mensaje claro en la casilla superior (ejemplo: `"Solucion de la tarea 1"`) y presione el botón **Commit** (o `Ctrl + Enter`).
- 5) Haga clic en el botón azul **Sync Changes** (o **Push**) para subir los cambios a su repositorio personal en GitHub.

4. Realizar la Entrega Formal (Pull Request)

Un **Pull Request (PR)** es la notificación mediante la cual usted solicita a la persona dueña del repositorio que revise los cambios que realizó en su copia.

- 1) Ingrese desde el navegador a la página web de **su Fork** en GitHub.
- 2) Verá un aviso indicando que su repositorio tiene cambios por delante del original *This branch is 1 commit....* Haga clic en **Contribute** y luego en **Open pull request**.
- 3) Verifique los cambios que realizó, escriba la descripción y haga clic en **Create pull request**.

- 4) En el título escriba su **Nombre Completo** (ejemplo: Entrega Taller 1 – Leonel Messi).
- 5) Haga clic en el botón verde **Create pull request** para completar su entrega.

Con estos pasos ya queda lista la entrega de su tarea y solo resta que yo revise los cambios y deje comentarios o sugerencias directamente en GitHub. Sin embargo, en el transcurso del semestre se irá actualizando el repositorio del curso y ustedes deben actualizar su copia local, pues esto no se hace automáticamente. ¿Cómo se hace esto?

5. Sincronizar su Fork para actualizar cambios

¡PRECAUCIÓN! Antes de descargar las actualizaciones, asegúrese de guardar y hacer **Commit** de sus cambios locales en VS Code. Si intenta hacer un *Pull* teniendo archivos modificados localmente sin confirmar, Git bloqueará la sincronización o generará un conflicto de fusión (*Merge Conflict*).

Para sincronizar su entorno de forma segura, siga estos pasos:

- 1) **Asegurar cambios locales:** En VS Code, vaya al panel de **Control de código fuente** (Ctrl + Shift + G), guarde sus avances, agregue los archivos con + y realice un **Commit** con un mensaje descriptivo.

- *Alternativa rápida:* Si no quiere guardar aún un commit de sus cambios locales incompletos, puede pausarlos ejecutando en la terminal:

```
git stash
```

- 2) **Sincronizar en GitHub (Nube):** Ingrese desde el navegador a la página web de su **Fork**. En la parte superior verá un aviso indicando que su rama está desactualizada. Haga clic en el botón **Sync fork** y luego seleccione **Update branch**.
- 3) **Descargar a VS Code (Local):** Vaya a VS Code y haga clic en el botón **Sync Changes** (o ejecute `git pull` en la terminal).
- 4) **Restaurar avances (si usó Stash):** Si pausó sus cambios en el paso 1, devuélvalos a su espacio de trabajo ejecutando:

```
git stash pop
```

Problemas:

1. Configuración de Entorno y Datos Personales

- (a) Dentro de la carpeta raíz de su proyecto clonado, cree una nueva carpeta llamada `Estudiantes/`.
- (b) Dentro de dicha carpeta, cree un archivo de texto plano nombrado con su usuario de GitHub, por ejemplo: `longas14.txt`.
- (c) En la primera línea del archivo, escriba su ****Nombre Completo****, ****Documento de Identidad**** y su ****Correo Institucional****.
- (d) En las siguientes líneas, redacte un breve párrafo (de 3 a 5 líneas) respondiendo: *¿Qué expectativa tiene sobre la aplicación de la física computacional en su formación académica?*.
- (e) Guarde el archivo, realice el proceso de **Add** y haga un ****Commit**** con el mensaje: "Se añade información de estudiante: su_usuario".

2. Una computadora solo sabe sumar, restar, multiplicar o dividir. Por esta razón, el cálculo de muchas funciones complicadas en física y matemática (como $\sin x$, $\cos x$ o e^x) se realiza mediante el uso de series de potencias (series de Taylor). Para la función coseno, la expansión alrededor de $x = 0$ está dada por la suma infinita:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}$$

donde el ángulo x está expresado en radianes. Dado que un sistema computacional no puede sumar infinitos términos, en la práctica la serie se trunca sumando únicamente los primeros N términos, asumiendo que el residuo es cuantitativamente despreciable.

- (a) Cree una carpeta con su nombre y primer apellido, sin espacios (ejemplo: `cristiano_ronaldo/`) dentro del directorio `Tareas/`
- (b) Dentro de `cristiano_ronaldo/` cree un archivo de texto o markdown llamado `pseudocodigo_coseno.txt` (o `.md`).
- (c) Elabore el pseudocódigo estructurado para aproximar $\cos(x)$ sumando los primeros N términos de la serie. *Sugerencia:* Estudie de nuevo la clase 3.
- (d) Verifique con `git status` que el archivo creado aparezca en el panel de control de código fuente.
- (e) Realice el ****Add****, ejecute un ****Commit**** con el mensaje "Solucion del taller 0: Pseudocodigo serie de Taylor para coseno" y haga el ****Push / Sync Changes**** hacia su Fork en GitHub.
- (f) Ingrese a la interfaz web de su Fork y realice el **Pull Request** formal hacia el repositorio del curso con el título: Entrega Taller 0 - [Su Nombre Completo].